Prota — Architecture Documentation

Version 1.0 · Architecture baseline · Spring Boot + PostgreSQL + React

Prepared for Saichu · October 20, 2026

1. Product summary

Prota is a responsive, installable web app (PWA) for tracking daily protein intake, with water and weight tracking, AI-assisted food logging, reminders, and a full analytics suite. Tagline: *Know your intake. Own your progress.*

Confirmed decisions

Area	Decision
App name	Prota
Logo	Concept A: circular progress ring with a geometric P, emerald accent
Theme	Light and dark modes, user-selectable, preference persisted
Frontend	React + TypeScript + Vite, Tailwind CSS with theme variables, Lucide icons, Recharts
Backend	Java + Spring Boot (Spring Security, Spring Data JPA, Bean Validation, Flyway)
Database	PostgreSQL (only required database)
AI	Google Gemini API, free tier initially, called only from the backend
Mobile	Mobile-first responsive PWA; not a native app
Source control	Git + GitHub

Architecture decisions

- Historical record policy: dated history is preserved. Individual entries can be edited or deleted in a controlled way. Old goal snapshots are never rewritten when current goals change.

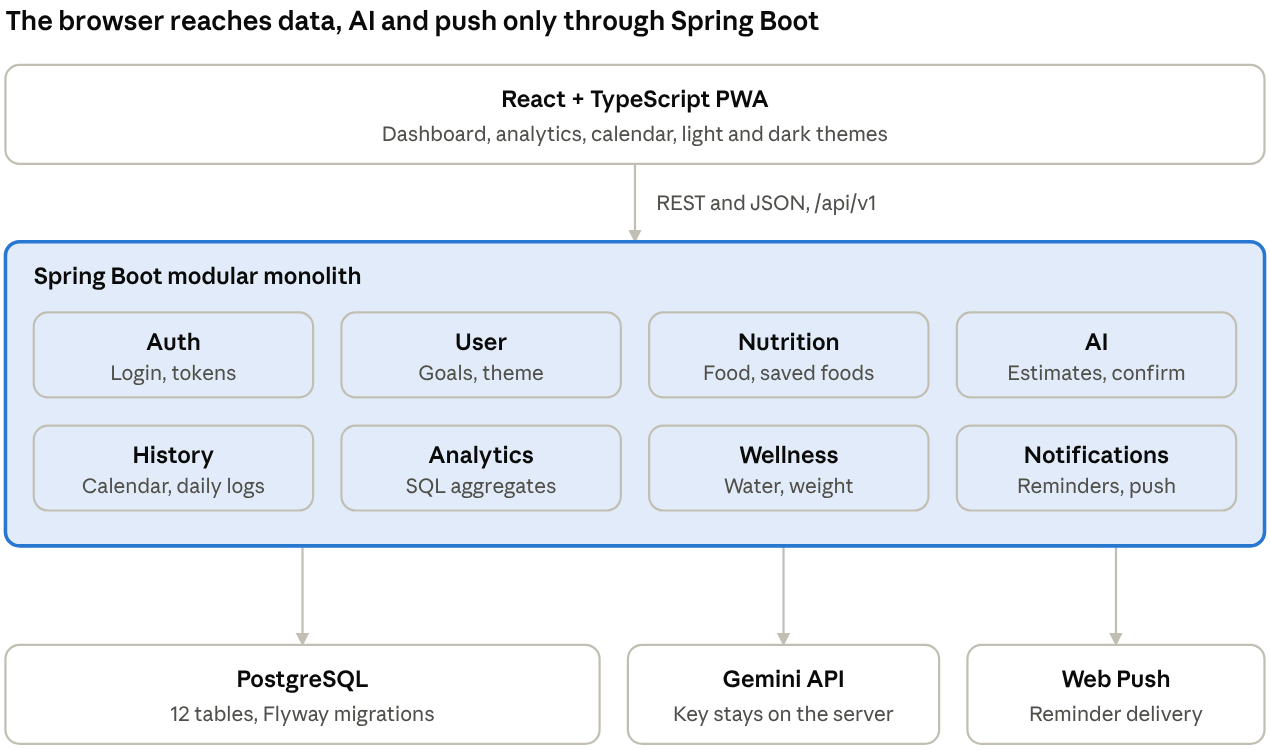
- Analytics experience: dashboard summaries plus a dedicated analytics page with advanced filters.

2. Design principles

1. **Historical integrity.** Each day stores its own goal snapshot. Changing today's goal never alters past results.
2. **Server-authoritative data.** PostgreSQL is the source of truth, Spring Boot validates and aggregates, React presents. The client never sends totals.
3. **Analytics as a first-class feature.** Charts are computed with SQL aggregations, not in the browser.
4. **AI proposes, the user confirms.** Gemini estimates never count toward intake until confirmed.
5. **Missing is not zero.** Days without tracking are gaps, not zero-intake days.
6. **Mobile-first.** Small screens first, scale up with breakpoints. No horizontal overflow, touch-friendly controls, bottom navigation on mobile and sidebar on larger screens, accessible contrast and focus states.

3. System architecture

Prota is a **modular monolith**: one Spring Boot application, one PostgreSQL database, one React frontend. Code is organised by feature module. No microservices, Redis, Kafka or separate analytics database in v1.



system architecture · React, Spring Boot, PostgreSQL, Gemini, Web Push

The browser talks only to the Spring Boot API, which is the single gateway to PostgreSQL, Gemini and Web Push.

Layer	Responsibility
React PWA	Dashboard, analytics page, calendar, forms, filters, pagination, themes, push registration
Spring Boot API	Authentication, validation, business rules, analytics, Gemini adapter, reminder scheduler
PostgreSQL	Relational records, constraints, indexes, aggregations
Gemini API	Text and image food estimation (backend only; key in environment variables)
Web Push	Browser notifications for reminders

Backend modules

Module	Responsibilities
Auth	Registration, login, logout, token refresh, session security
User	Profile, settings, goals, theme preferences
Nutrition	Food entries, saved foods, daily summaries
AI	Gemini integration, estimates, schema validation, confirmation
History	Calendar queries and date-wise records
Analytics	Aggregated chart data, filters, date grouping
Wellness	Weight and water records
Notifications	Reminder schedules, push delivery, delivery log

4. Database design

4.1 Conventions

- Primary keys: UUID.
- Timestamps: TIMESTAMPTZ, stored in UTC.
- Calendar days: DATE, interpreted using the user's saved IANA timezone.
- Protein: NUMERIC(8,2) grams. Calories: NUMERIC(9,2) kcal. Weight: NUMERIC(7,3) kg. Water: integer millilitres.
- All relationships use foreign keys. Every user-owned query is scoped to the authenticated user.
- Enum-like values are validated both in the database (CHECK) and in Java.

4.2 The 12 tables

1. users: account identity.

Column	Type	Rules
id	UUID	PK

Column	Type	Rules
full_name	VARCHAR(100)	NN
email	VARCHAR(254)	NN, UQ, normalized
password_hash	VARCHAR(255)	NN (Argon2id or Spring Security encoder)
status	VARCHAR(20)	NN, default ACTIVE (ACTIVE, SUSPENDED, PENDING_VERIFICATION)
email_verified_at	TIMESTAMPTZ	Nullable
created_at, updated_at	TIMESTAMPTZ	NN

2. user_settings: current preferences and goals (one row per user, not historical).

Column	Type	Rules
user_id	UUID	PK, FK to users
protein_goal_g	NUMERIC(8,2)	NN, positive
calorie_goal_kcal	NUMERIC(9,2)	Nullable, positive
water_goal_ml	INTEGER	Nullable, positive
weight_unit	VARCHAR(5)	NN, KG or LB
timezone	VARCHAR(64)	NN, valid IANA zone
theme	VARCHAR(10)	NN, LIGHT, DARK, SYSTEM
updated_at	TIMESTAMPTZ	NN

3. daily_logs: one record per user per date, holding the goal snapshot.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
log_date	DATE	NN

Column	Type	Rules
timezone_at_creation	VARCHAR(64)	NN
protein_goal_g	NUMERIC(8,2)	NN, positive (snapshot)
calorie_goal_kcal	NUMERIC(9,2)	Nullable (snapshot)
water_goal_ml	INTEGER	Nullable (snapshot)
created_at, updated_at	TIMESTAMPTZ	NN

UNIQUE (user_id, log_date). Created lazily on first activity for that date. Snapshots are immutable through normal application operations.

4. food_entries: confirmed food eaten on a day.

Column	Type	Rules
id	UUID	PK
daily_log_id	UUID	FK to daily_logs, NN
saved_food_id	UUID	Nullable FK to saved_foods
ai_estimate_id	UUID	Nullable FK to ai_food_estimates
food_name	VARCHAR(200)	NN
meal_type	VARCHAR(20)	NN (BREAKFAST, LUNCH, DINNER, SNACK, OTHER)
category	VARCHAR(40)	Nullable
quantity	NUMERIC(10,3)	NN, positive
quantity_unit	VARCHAR(20)	NN
protein_g	NUMERIC(8,2)	NN, non-negative
calories_kcal	NUMERIC(9,2)	Nullable, non-negative
carbohydrates_g, fat_g	NUMERIC(8,2)	Nullable, non-negative
nutrition_source	VARCHAR(30)	NN (MANUAL, AI_CONFIRMED, SAVED_FOOD, EXTERNAL_DATABASE)

Column	Type	Rules
notes	VARCHAR(500)	Nullable
consumed_at	TIMESTAMPTZ	NN
created_at, updated_at	TIMESTAMPTZ	NN

Values are snapshots: editing a saved food never changes existing entries. The app validates that referenced saved foods and estimates belong to the same user.

5. saved_foods: reusable foods and custom nutrition profiles.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
name	VARCHAR(200)	NN
brand	VARCHAR(120)	Nullable
serving_quantity	NUMERIC(10,3)	NN, positive
serving_unit	VARCHAR(20)	NN
protein_g	NUMERIC(8,2)	NN, non-negative
calories_kcal, carbohydrates_g, fat_g	NUMERIC	Nullable
external_source, external_food_id	VARCHAR	Nullable
created_at, updated_at	TIMESTAMPTZ	NN

6. water_entries: one row per water intake event.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
entry_date	DATE	NN (local date)

Column	Type	Rules
amount_ml	INTEGER	NN, positive, sensible maximum
consumed_at	TIMESTAMPTZ	NN
created_at	TIMESTAMPTZ	NN

7. weight_entries: dated measurements.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
weight_kg	NUMERIC(7,3)	NN, positive (canonical kg)
recorded_at	TIMESTAMPTZ	NN
entry_date	DATE	NN
notes	VARCHAR(500)	Nullable
created_at, updated_at	TIMESTAMPTZ	NN

Multiple measurements per day are allowed. Trends use the latest measurement of each local date.

8. ai_food_estimates: AI proposals awaiting confirmation.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
input_type	VARCHAR(10)	NN, TEXT or IMAGE
original_text	TEXT	Nullable
image_reference	VARCHAR(500)	Nullable (private storage only, never image bytes in the database)
model_name	VARCHAR(120)	NN

Column	Type	Rules
status	VARCHAR(20)	NN (PENDING, CONFIRMED, REJECTED, FAILED, EXPIRED)
estimate_json	JSONB	NN
error_code	VARCHAR(80)	Nullable
created_at	TIMESTAMPTZ	NN
confirmed_at, expires_at	TIMESTAMPTZ	Nullable

9. reminder_settings: independent reminder preferences.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
reminder_type	VARCHAR(20)	NN (PROTEIN, WATER, WEIGHT)
enabled	BOOLEAN	NN
schedule_type	VARCHAR(20)	NN (FIXED_TIMES, INTERVAL, SELECTED_DAYS)
scheduled_times	JSONB	NN
interval_minutes	INTEGER	Nullable
timezone	VARCHAR(64)	NN
quiet_hours_start, quiet_hours_end	TIME	Nullable
created_at, updated_at	TIMESTAMPTZ	NN

UNIQUE (user_id, reminder_type).

10. push_subscriptions: browser push endpoints.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
endpoint	TEXT	NN, UQ
p256dh_key, auth_key	TEXT	NN (sensitive)
device_label	VARCHAR(100)	Nullable
last_success_at	TIMESTAMPTZ	Nullable
created_at, updated_at	TIMESTAMPTZ	NN

11. notification_events: scheduled notifications and delivery attempts.

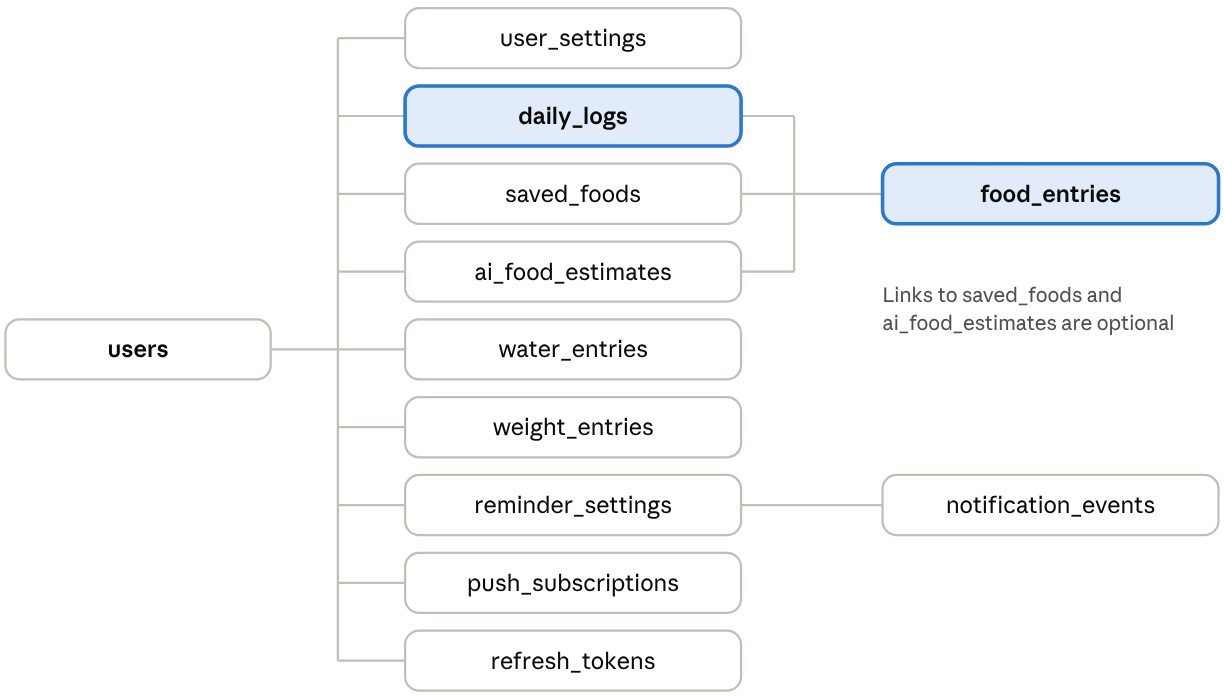
Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
reminder_setting_id	UUID	Nullable FK to reminder_settings
event_type	VARCHAR(30)	NN
scheduled_at	TIMESTAMPTZ	NN
status	VARCHAR(20)	NN (SCHEDULED, PROCESSING, SENT, FAILED, CANCELLED, SKIPPED)
deduplication_key	VARCHAR(200)	NN, UQ
attempt_count	INTEGER	NN, default 0
sent_at, last_attempt_at	TIMESTAMPTZ	Nullable
error_code	VARCHAR(80)	Nullable
created_at	TIMESTAMPTZ	NN

12. refresh_tokens: session renewal and revocation.

Column	Type	Rules
id	UUID	PK
user_id	UUID	FK to users, NN
token_hash	VARCHAR(255)	NN, UQ (hash only, never the token)
family_id	UUID	NN (for reuse detection)
device_label	VARCHAR(100)	Nullable
expires_at	TIMESTAMPTZ	NN
revoked_at, last_used_at	TIMESTAMPTZ	Nullable
created_at	TIMESTAMPTZ	NN

4.3 Relationships and deletion rules

Every record traces to one user; food entries hang off daily logs



entity relationships · 12 tables

All tables are owned by users. Notification events also belong to the user directly, and each daily log carries that day's goal snapshot.

Parent	Child	Relationship
users	user_settings	One-to-one
users	daily_logs, saved_foods, water_entries, weight_entries, ai_food_estimates, reminder_settings, push_subscriptions, notification_events, refresh_tokens	One-to-many
daily_logs	food_entries	One-to-many
saved_foods, ai_food_estimates	food_entries	Optional reference
reminder_settings	notification_events	Optional reference

- Deleting a food entry removes only that entry. The daily log and its goal snapshot stay.
- Saved-food templates never cascade-delete historical entries. Prefer deactivation when referenced.
- AI estimates referenced by confirmed entries are not hard-deleted independently.
- Foreign keys use RESTRICT or SET NULL where history must survive. Cascade only where data loss is intended.
- Account deletion goes through an explicit data-deletion workflow.

4.4 Indexes and constraints

Table	Index or constraint	Purpose
users	Unique normalized email	Login, duplicate prevention
daily_logs	Unique (user_id, log_date)	One record per user per date; also serves calendar lookups
food_entries	(daily_log_id, consumed_at DESC, id DESC)	Per-day history with stable ordering
food_entries	(ai_estimate_id)	Estimate lookups
saved_foods	(user_id, name)	Search

Table	Index or constraint	Purpose
water_entries	(user_id, entry_date, consumed_at DESC)	Daily totals
weight_entries	(user_id, entry_date DESC, recorded_at DESC)	History and trends
ai_food_estimates	(user_id, created_at DESC); (status, expires_at)	History; cleanup
reminder_settings	Unique (user_id, reminder_type)	One config per type
push_subscriptions	Unique (endpoint)	No duplicates
notification_events	Unique (deduplication_key); (status, scheduled_at); (user_id, scheduled_at DESC)	Dedup; scheduler polling; history
refresh_tokens	Unique token_hash; (user_id, expires_at)	Lookup; cleanup

Add CHECK constraints for positive goals and quantities and non-negative nutrition values. Verify composite indexes with EXPLAIN ANALYZE on real filter combinations before adding more.

5. REST API contract

All endpoints use the `/api/v1` prefix and JSON (multipart only for image upload). Every user-owned endpoint requires authentication and ownership checks.

5.1 Auth and settings

Method	Endpoint	Behaviour
POST	/auth/register	Create account
POST	/auth/login	Authenticate
POST	/auth/refresh	Rotate refresh token
POST	/auth/logout	Revoke session
GET, PATCH	/users/me	Profile

Method	Endpoint	Behaviour
GET, PATCH	/settings	Goals and preferences (updates current settings only, never old snapshots)

5.2 Nutrition

Method	Endpoint	Behaviour
GET	/daily-logs/{date}	Daily summary
GET	/daily-logs?from=&to=	Date-wise history
POST	/daily-logs/{date}/foods	Add food (creates the daily log if needed, in one transaction)
GET	/food-entries	Filtered, paginated history
GET, PATCH, DELETE	/food-entries/{id}	Read, edit, delete one entry
GET, POST	/saved-foods	Search, create
PATCH, DELETE	/saved-foods/{id}	Update, delete safely
GET	/calendar?month=YYYY-MM	One compact summary per day

Example food creation request:

```
{
  "foodName": "Chicken breast",
  "mealType": "LUNCH",
  "category": "MEAT",
  "quantity": 150,
  "quantityUnit": "G",
  "proteinG": 46.5,
  "caloriesKcal": 247,
  "nutritionSource": "MANUAL",
  "consumedAt": "2026-10-20T12:30:00+05:30"
}
```

5.3 AI estimation

Method	Endpoint	Behaviour
POST	/ai/food-estimates	Estimate from text
POST	/ai/food-estimates/image	Estimate from image
GET	/ai/food-estimates/{id}	Retrieve estimate
POST	/ai/food-estimates/{id}/confirm	Save proposed foods (idempotent, single transaction)
POST	/ai/food-estimates/{id}/reject	Reject

AI output is validated against a strict schema and treated as untrusted input. Nutrition databases (FoodData Central, Open Food Facts) supply protein values when a suitable match exists; the AI identifies foods and quantities.

5.4 Water, weight, reminders

Method	Endpoint	Behaviour
GET, POST	/water, /weight	List (paginated) and add
PATCH, DELETE	/water/{id}, /weight/{id}	Edit, delete
GET	/reminders	All reminder preferences
PUT	/reminders/{type}	Replace one configuration
POST, DELETE	/push-subscriptions	Register, unregister a browser
GET	/notifications	Paginated history

6. Pagination, filtering and errors

Pagination

- Page numbering starts at 0. Default size 20, maximum 100 (enforced server-side).
- Sort fields are allowlisted. A unique secondary sort key (`id`) guarantees stable ordering.
- Changing any filter resets the page to 0. Filters and sort persist while paging.

- Invalid page sizes, sort fields or dates return a validation error.

Screen	Default size	Sort
Food history	20	Newest first
A day's food entries	10	Most recently added first
Weight, water, notification history	20	Newest first
Saved foods	20	Name or recently used

Standard envelope:

```
{
  "content": [],
  "page": 0,
  "size": 20,
  "totalElements": 145,
  "totalPages": 8,
  "first": true,
  "last": false,
  "hasNext": true
}
```

Filtering supports combinations: date range, food name search, meal type, category, protein range, calorie range, nutrition source, and sorting by date, protein, calories or name. Example:

`GET /api/v1/food-entries?from=2026-10-01&to=2026-10-20&mealType=LUNCH&minProteinG=10&page=0&size=20&sort=consumedAt,desc`

Errors use RFC 9457 Problem Details with a stable `code` and field-level `errors`.

Status	Meaning
400	Invalid request or parameter
401 / 403	Unauthenticated / not permitted
404	Not found or not accessible
409	Duplicate or conflicting operation

Status	Meaning
422	Valid structure, unacceptable business state
429	Rate limit exceeded
500 / 502 / 503	Server error / AI or dependency unavailable

Never expose stack traces, SQL errors or API keys in responses.

7. Analytics

Analytics is its own Spring module and its own frontend page, plus summary cards on the dashboard. Endpoints return chart-ready aggregates, never raw records.

Endpoint	Output
GET /analytics/overview?from=&to=	KPI summary cards
GET /analytics/protein-trend?from=&to=&groupBy=	Line chart series
GET /analytics/protein-sources?from=&to=&groupBy=	Pie or donut categories
GET /analytics/goal-achievement?from=&to=	Goal completion bars and rates
GET /analytics/weight-trend?from=&to=	Weight series
GET /analytics/water-trend?from=&to=	Daily water totals
GET /analytics/food-breakdown?from=&to=&mealType=	Protein and calories by food or meal

Filters: presets (7 days, 30 days, 90 days, 6 months, 1 year, custom), grouping (day, week, month), food category, meal type, nutrition source, achieved versus below-goal, and optional comparison with the previous equivalent period.

Calculation rules

Metric	Rule
Daily protein	Sum of confirmed food-entry protein for the local date. Pending AI estimates never count.
Goal achieved	Daily protein is at least that day's stored goal snapshot, never the current setting.
Completion rate	100 × days achieved ÷ eligible days.
Average daily protein	Total protein ÷ logged days (denominator labelled in the UI).
Best day / frequent foods	Highest daily total; entry counts over the selected period.
Streak	Consecutive dates meeting the stored goal. A missing date breaks the streak.
Weight change	Last minus first representative measurement in the range, with no interpolation.
Water average	Total water ÷ days with water records.

Edge cases

- No data versus zero: gaps are shown or labelled, and missing days are excluded from rates.
- Calories include only entries with known values; show coverage or mark totals incomplete.
- Entries without a category appear as Uncategorized in pie charts. If the total is zero, return an empty state, not a division by zero.
- Convert timestamps to the user's local calendar before grouping.
- Date ranges are inclusive with a configurable maximum. Weeks run Monday to Sunday everywhere.
- Edits and deletions reflect in aggregates immediately.

Calendar day states: goal achieved; tracked but goal missed; no tracking data; tracked with an incomplete metric.

8. Transactions and consistency

Operation	Required behaviour
Add food	Create the daily log if needed and save the entry in one transaction
Edit / delete food	Verify ownership; delete only that entry, keep the log and snapshot
Confirm AI estimate	Create all entries and mark the estimate confirmed atomically; idempotent
Change goals	Update current settings only
Record weight / water	Save a dated entry without overwriting older ones
Refresh session	Rotate token and revoke the previous one atomically
Schedule notification	Persist a deduplicated event before delivery

Use optimistic locking (a `version` column) on mutable records if concurrent edits from two sessions must be detected. If audit history becomes a requirement, add it in a later migration.

9. Security and performance

- Spring Security with short-lived access tokens and rotating, hashed refresh tokens with reuse detection.
- Scope every query to the authenticated user. Test cross-user access explicitly.
- DTOs, request validation and centralised exception handling. Document the API.
- Gemini key and all secrets in backend environment variables, never in React or the repository.
- Never log credentials or private AI uploads. Store images only in private storage and delete them when no longer needed.
- Rate-limit AI and authentication endpoints.
- Flyway migrations for every schema change.
- Tests for date boundaries, timezone grouping, goal changes, analytics calculations, pagination and cross-user access.

10. Frontend rules

- Mobile-first CSS with responsive breakpoints. Bottom navigation on mobile, sidebar on larger screens.
- No horizontal overflow: tables, calendars, forms and charts adapt to narrow screens.
- Touch-friendly controls, visible focus states, keyboard support, sufficient contrast.
- Light and dark themes via CSS variables; the choice is saved in user settings.
- Charts show loading, empty, error and filtered states.
- PWA manifest and service worker for installability and web push.

11. Development phases

Phase	Scope
1. Foundation	Spring Boot project, PostgreSQL, Flyway migrations, authentication, profile and settings
2. Nutrition and history	Daily logs, food entries, saved foods, goal snapshots, calendar, filtering and pagination
3. Analytics	SQL aggregations, chart APIs, date filters, goal achievement, weight trends
4. AI and wellness	Gemini estimates with confirmation workflow, water and weight tracking
5. Reminders and release	Push subscriptions, scheduler, responsive polish, integration tests, deployment

12. Open items

- Verify current Gemini free-tier quotas and data-use terms for the chosen model before building the AI module.
- Check domain, trademark and app-store availability for the name Prota.
- Produce the final logo exports and colour palette.
- Next deliverables: detailed ER diagram, Flyway migration plan and Spring Boot package structure, before any implementation code.

13. Final word

Prota's architecture is settled. Twelve tables hold everything, and the rules that matter are fixed: past goals are never rewritten, the server owns every total, AI suggestions count only after confirmation, and missing data is never treated as zero. What remains is building it in phase order, starting with the foundation.